

Scientific Forgery Image Detection

By: Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam

May 8, 2026

Software Engineering Tools, CS 3338

Table of Contents

1. Introduction
2. System Architecture
3. User Interface Design
4. Glossary
5. Versions Table
6. References

1. Introduction

1.1 Purpose

Scientific images play an important role in academic research and published studies. However, some images may be manipulated through techniques such as copy-move forgery, where parts of an image are duplicated and placed elsewhere to falsify results.

The purpose of this system is to detect and analyze copy-move forgery in biomedical and scientific images. Users will be able to upload images through a web interface, and the system will process them to identify potentially duplicated regions. This helps support scientific integrity by assisting researchers and reviewers in identifying manipulated figures.

1.2 Intended Audience

This document is intended for:

- Software developers working on the system
- Course instructors evaluating the project
- Students involved in system design
- Future maintainers of the software

1.3 Overview

The system is a web-based application consisting of a frontend interface and a backend server. Users upload images through the frontend, and the backend processes these images for future detection logic development.

At this stage, the system focuses on:

- Project setup
- Frontend structure
- Backend structure
- Basic workflow definition
- Documentation planning

2. System Architecture

2.1 Overview

The system follows a simple client-server architecture.

2.2 Frontend

The frontend is responsible for user interaction and was **partially implemented in Snapshot 2**.

Responsibilities:

- Provide image upload interface (implemented in Snapshot 2)
- Send image data to backend (prepared for future integration)
- Display backend responses (placeholder at this stage)

Technologies:

- HTML
- React

2.3 Backend

The backend handles incoming requests from the frontend.

Responsibilities:

- Receive uploaded images
- Process API requests
- Return placeholder or future detection results

Technologies:

- Python
- FastAPI
- Uvicorn

2.4 Workflow Design

Basic Workflow (Updated in Snapshot 2 Progress):

- User uploads image (frontend implemented)
- Frontend sends image structure prepared for backend communication
- Backend receives request (to be fully integrated in next snapshot)
- System processes image (future detection logic)
- Response is returned to frontend
- User views result

3. User Interface Design

3.1 Homepage

The homepage introduces the application and provides navigation to upload functionality.

Elements:

- Project title
- Short description
- Navigation to upload page

3.2 Image Upload Interface (Implemented in Snapshot 2).

Elements:

- Upload button
- File selection option
- Submit button
- Basic image preview structure (prepared/implemented)

3.3 Results Display (Basic Placeholder)

- Confirmation of upload
- Placeholder response from backend
- Structure prepared for future detection output display

4. Glossary

Term	Definition
API	Application Programming Interface used for communication between frontend and backend
Backend	Server-side system that processes requests
Frontend	User interface of the application
Copy-Move Forgery	Image manipulation where parts of an image are duplicated within the same image
React.js	JavaScript library used for building user interfaces
FastAPI	Python framework used for building APIs
Uvicorn	Server used to run FastAPI applications

5. Versions Table

Version	Date	Description	Authors
1.0	May 8, 2026	Initial Software Design Document created for Snapshot 1	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam
1.1	May 10, 2026	Added frontend implementation progress and updated system architecture and UI design to reflect image upload interface development	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam